Innominatum CONSULTORIA

Docker Compose

ServiceNow - Deloitte

Prof. Tiago Rodrigues

IPCA POLITÉCNICO DO CÁVADO E DO AVE
INSTITUTO DO EMPREGO E FORMAÇÃO PROFISSIONAL
PRR Plano de Recuperação e Resiliência
REPÚBLICA PORTUGUESA
Financiado pela União Europeia NextGenerationEU
IAPMEI Parcerias para o Crescimento
PORTUGAL DIGITAL

Objetivo

Perceber como o Docker Compose orquestra múltiplos contentores como um único serviço, e saber configurar ambientes distintos através de variáveis de ambiente.

Objetivos Específicos

Ao final da sessão, os formandos deverão ser capazes de:

- Explicar o propósito do Docker Compose e quando utilizá-lo em vez de contentores isolados;
- Ler e escrever um ficheiro docker-compose.yml com services, volumes, networks e depends_on;
- Usar os comandos essenciais para operar e depurar o ambiente.

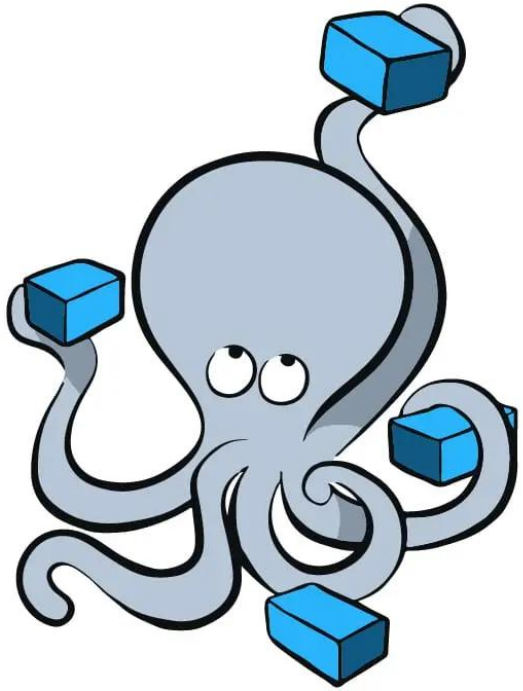
Innominatum CONSULTORIA

Docker Compose

Orquestrar múltiplos contentores como um único serviço

IPCA POLITÉCNICO DO CÁVADO E DO AVE
INSTITUTO DO EMPREGO E FORMAÇÃO PROFISSIONAL
PRR Plano de Recuperação e Resiliência
REPÚBLICA PORTUGUESA
Financiado pela União Europeia NextGenerationEU
IAPMEI Parcerias para o Crescimento
PORTUGAL DIGITAL

Docker Compose



docker
Compose

Quando um Contentor Não Chega

Na aula anterior aprendemos a empacotar a aplicação Angular num contentor. Mas uma aplicação real não existe só de frontend.

Numa arquitetura típica temos: o frontend Angular, uma API backend (json-server, Node, etc.), uma base de dados, e por vezes um proxy. São processos separados.

Quando um Contentor Não Chega

Gerir cada contentor manualmente é impraticável: arrancar na ordem certa, garantir que um espera pelo outro, partilhar a mesma rede interna, gerir volumes.

O Docker Compose resolve exatamente isto: descreve todos os serviços, as suas relações e configurações num único ficheiro, e arranca tudo com um só comando.

O que é o Docker Compose

Docker Compose é uma ferramenta que permite definir e correr aplicações multi-contentor através de um ficheiro YAML chamado docker-compose.yml.

O que é o Docker Compose

Com um único ficheiro descrevemos quantos serviços existem, que imagem usa cada um, que portas serão expostas, que volumes monta, de que outros serviços depende.

O Compose é ideal para desenvolvimento local e ambientes de teste. Em produção a orquestração à escala usa Kubernetes, mas os conceitos são os mesmos.

Anatomia do docker-compose.yml

O ficheiro é escrito em YAML e tem uma estrutura fixa com chaves de topo que organizam tudo o que o Compose precisa de saber.

Anatomia do docker-compose.yml

```yaml
services:
  postgres:
    image: postgres:18.1-bookworm
    container_name: postgres
    environment:
      POSTGRES_PASSWORD: "${POSTGRES_PASSWORD}"
    ports:
      - "5433:5432"
    volumes:
      - postgres_data:/var/lib/postgresql
    restart: unless-stopped

  pgadmin:
    image: dpage/pgadmin4:latest
    container_name: pgadmin
    environment:
      PGADMIN_DEFAULT_EMAIL: "${PGADMIN_EMAIL}"
      PGADMIN_DEFAULT_PASSWORD: "${PGADMIN_PASSWORD}"
    ports:
      - "15432:80"
    depends_on:
      - postgres
    restart: unless-stopped

volumes:
  postgres_data:
```

Anatomia do docker-compose.yml

- **version**: declara a versão da especificação Compose (ex: "3.9"). Determina que funcionalidades estão disponíveis;
- **services**: bloco principal. Cada entrada é um serviço (contentor). Dentro de cada serviço definimos a imagem ou o *build*, as portas, os volumes e as dependências;

Anatomia do docker-compose.yml

- **volumes**: declara volumes nomeados partilhados entre serviços ou persistidos no host. Útil para bases de dados;
- **networks**: por omissão o Compose cria uma rede interna. Podemos declarar redes adicionais para segmentar o tráfego entre serviços.

Configuração de um Serviço

Cada serviço dentro de services aceita estas chaves:

- **build**: caminho para o Dockerfile (em vez de image). Usa a imagem local construída no momento do up –build;
- **image**: imagem a usar diretamente do registry (ex: nginx:alpine, node:20-alpine, json-server);

Configuração de um Serviço

- **ports**: mapeamento de portas no formato "host:contentor" (ex: "4200:80" expõe a porta 80 do contentor na 4200 do host).
- **environment**: lista de variáveis de ambiente a injetar no contentor (ex: API_URL=http://api:3000).
- **depends_on**: lista de serviços que devem estar ativos antes deste arrancar (ex: depends_on: [api]).
- **volumes**: monta diretórios do host ou volumes nomeados (ex: ./db.json:/app/db.json para json-server).

Configuração de um Serviço

- **restart**: **always** garante que o contentor reinicia automaticamente se falhar. Útil para serviços críticos.
- **healthcheck**: permite definir um comando que verifica se o serviço está realmente pronto (além de apenas a correr).

Redes e Comunicação Interna

Como os serviços falam entre si:

- O Compose cria automaticamente uma rede interna isolada. Os serviços comunicam pelo nome do serviço.

Exemplo: o serviço app pode chamar http://api:3000 para aceder ao serviço api, sem expor essa porta ao exterior.

- Só as portas declaradas em *ports* ficam acessíveis fora do Docker (do browser do host). As restantes são internas.

Variáveis de Ambiente

.env, .env.dev e .env.prod

Configuração por Contexto

Uma aplicação real comporta-se de forma diferente em desenvolvimento e em produção: URLs de API, níveis de log, funcionalidades ativas.

Configuração por Contexto

O padrão com Docker Compose é usar ficheiros .env separados por contexto.

- O .env é o ficheiro padrão lido automaticamente.
- .env.dev e .env.prod são carregados explicitamente: docker compose --env-file .env.prod up.
- O Compose substitui as variáveis no docker-compose.yml.

Configuração por Contexto

Regra fundamental:

- Nenhum ficheiro .env vai para o repositório Git.
- Só o .env.example (com chaves mas sem valores) é versionado como documentação.

Boas Práticas de Configuração

- Nunca colocar segredos (passwords, tokens, chaves API) em texto no docker-compose.yml. Usar sempre variáveis de ambiente.
- Adicionar .env, .env.dev e .env.prod ao .gitignore. Versionar apenas .env.example como documentação das chaves necessárias.

Boas Práticas de Configuração

- No Angular as variáveis de ambiente do Compose são para o servidor nginx ou Node. As variáveis do environment.ts são compiladas no bundle.
- Para segredos em produção usar Docker Secrets ou o gestor de segredos da plataforma cloud (AWS Secrets Manager, Azure Key Vault, etc.).
- Manter um serviço por responsabilidade: um contentor = um processo. Não colocar frontend e backend no mesmo contentor.

Comandos Essenciais

up, down, logs e ps

Ambiente com Compose

- **docker compose up --build**: constrói (ou reconstrói) as imagens e arranca todos os serviços. Adicionar -d para modo background (detached).
- **docker compose down**: para e remove todos os contentores e a rede criada. Os volumes nomeados persistem por omissão (--volumes para apagar também).

Ambiente com Compose

- **docker compose logs -f [serviço]:** mostra os logs em tempo real. Sem especificar serviço mostra todos os serviços combinados.
- **docker compose ps:** lista o estado atual de todos os serviços (a correr, parado, a reiniciar).

Docker composse logs <nome_container>

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\tiago\Documents\innominatum\idp\pl\idp> docker logs angular-app
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/03/13 08:46:38 [notice] 1#1: using the "epoll" event method
2026/03/13 08:46:38 [notice] 1#1: nginx/1.29.6
2026/03/13 08:46:38 [notice] 1#1: built by gcc 15.2.0 (Alpine 15.2.0)
2026/03/13 08:46:38 [notice] 1#1: OS: Linux 6.6.87.2-microsoft-standard-WSL2
2026/03/13 08:46:38 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/03/13 08:46:38 [notice] 1#1: start worker processes
2026/03/13 08:46:38 [notice] 1#1: start worker process 30
2026/03/13 08:46:38 [notice] 1#1: start worker process 31
2026/03/13 08:46:38 [notice] 1#1: start worker process 32
2026/03/13 08:46:38 [notice] 1#1: start worker process 33
2026/03/13 08:46:38 [notice] 1#1: start worker process 34
2026/03/13 08:46:38 [notice] 1#1: start worker process 35
2026/03/13 08:46:38 [notice] 1#1: start worker process 36
2026/03/13 08:46:38 [notice] 1#1: start worker process 37
2026/03/13 08:46:38 [notice] 1#1: start worker process 38
2026/03/13 08:46:38 [notice] 1#1: start worker process 39
2026/03/13 08:46:38 [notice] 1#1: start worker process 40
2026/03/13 08:46:38 [notice] 1#1: start worker process 41
2026/03/13 08:46:38 [notice] 1#1: start worker process 42
2026/03/13 08:46:38 [notice] 1#1: start worker process 43
2026/03/13 08:46:38 [notice] 1#1: start worker process 44
2026/03/13 08:46:38 [notice] 1#1: start worker process 45
172.21.0.1 - - [13/Mar/2026:08:55:52 +0000] "GET / HTTP/1.1" 200 430 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:148.0) Gecko/20100101
Firefox/148.0" "-"
172.21.0.1 - - [13/Mar/2026:08:55:53 +0000] "GET /styles-5INURTSO.css HTTP/1.1" 200 0 "http://localhost:4200/" "Mozilla/5.0 (Windows NT 10.0;
 Win64; x64; rv:148.0) Gecko/20100101 Firefox/148.0" "-"
172.21.0.1 - - [13/Mar/2026:08:55:53 +0000] "GET /main-OA6GHU53.js HTTP/1.1" 200 214185 "http://localhost:4200/" "Mozilla/5.0 (Windows NT 10.
0; Win64; x64; rv:148.0) Gecko/20100101 Firefox/148.0" "-"
PS C:\Users\tiago\Documents\innominatum\idp\pl\idp>
```

Docker compose ps

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\tiago\Documents\innominatum\idp\pl\idp> docker compose ps
NAME          IMAGE            COMMAND                  SERVICE   CREATED          STATUS                    PORTS
angular-app   idp-app          "/docker-entrypoint.…"   app       11 minutes ago   Up 11 minutes             0.0.0.0:4200->80/tcp, [::]:4200-
>80/tcp
mock-api      node:20-alpine   "docker-entrypoint.s…"   api       6 hours ago      Up 11 minutes (healthy)   0.0.0.0:3001->3000/tcp, [::]:300
1->3000/tcp
PS C:\Users\tiago\Documents\innominatum\idp\pl\idp>
```

Ambiente com Compose

- **docker compose restart [serviço]:** reinicia apenas um serviço sem afetar os restantes. Útil para aplicar alterações de configuração.
- **docker compose exec [serviço] sh:** abre uma shell dentro de um contentor em execução. Equivalente ao docker exec mas com nome de serviço.

Innominatum CONSULTORIA

Caso Prático

Angular + json-server: um ambiente local completo

IPCA POLITÉCNICO DO CÁVADO E DO AVE
INSTITUTO DO EMPREGO E FORMAÇÃO PROFISSIONAL
PRR Plano de Recuperação e Resiliência
REPÚBLICA PORTUGUESA
Financiado pela União Europeia NextGenerationEU
IAPMEI Parcerias para o Crescimento
PORTUGAL DIGITAL

Frontend Angular + Mock API

O json-server é uma ferramenta que transforma um ficheiro db.json numa API REST completa com endpoints GET, POST, PUT e DELETE. Ideal para desenvolvimento.

Arquitetura do caso prático: serviço app (frontend Angular com nginx) e serviço api (json-server a servir dados de db.json).

Frontend Angular + Mock API

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

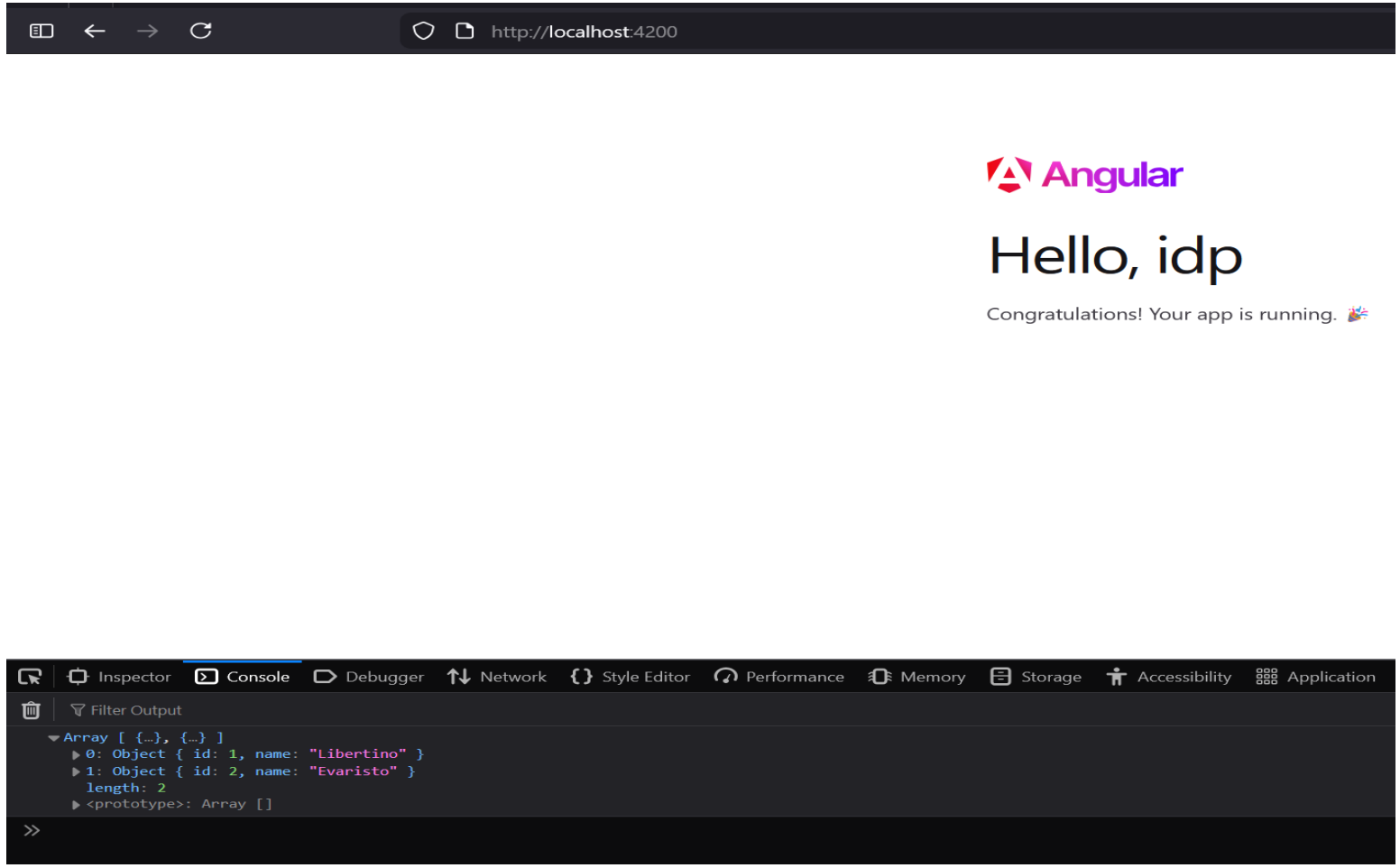
```
mock-api      | changed 120 packages in 9s
mock-api      |
mock-api      | 21 packages are looking for funding
mock-api      |   run `npm fund` for details
mock-api      |
mock-api      |   \{^_^}/ hi!
mock-api      |
mock-api      |   Loading db.json
mock-api      |   Done
mock-api      |
mock-api      |   Resources
mock-api      |   http://0.0.0.0:3000/users
mock-api      |
mock-api      |   Home
mock-api      |   http://0.0.0.0:3000
mock-api      |
mock-api      |   Type s + enter at any time to create a snapshot of the database

v View in Docker Desktop   o View Config   w Enable Watch   d Detach
```

Frontend Angular + Mock API

O serviço app depende do serviço api (depends_on: [api]). O Angular envia pedidos para http://localhost:3001 no browser, mas dentro da rede Docker usa http://api:3000.

O volume ./db.json:/app/db.json monta o ficheiro do host diretamente no contentor do json-server, permitindo editar os dados sem reconstruir a imagem.

Frontend Angular + Mock API



Frontend Angular + Mock API

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\tiago\Documents\innominatum\idp\pl\idp> curl http://localhost:3001/users


StatusCode        : 200
StatusDescription : OK
Content           : [
                      {
                        "id": 1,
                        "name": "Libertino"
                      },
                      {
                        "id": 2,
                        "name": "Evaristo"
                      }
                    ]
RawContent        : HTTP/1.1 200 OK
                    Vary: Origin, Accept-Encoding
                    Access-Control-Allow-Credentials: true
                    Pragma: no-cache
                    X-Content-Type-Options: nosniff
                    Connection: keep-alive
                    Keep-Alive: timeout=5
                    Content-Length...
Forms             : {}
Headers           : {[Vary, Origin, Accept-Encoding], [Access-Control-Allow-Credentials, true], [Pragma, no-cache],
                    [X-Content-Type-Options, nosniff]...}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : mshtml.HTMLDocumentClass
RawContentLength  : 93
```

Frontend Angular + Mock API

Containers / angular-app

angular-app

dfaaff854b0e idp-app:latest

4200:80

Logs | Inspect | Bind mounts | Exec | Files | Stats

```
/ # curl http://api:3000/users
[
  {
    "id": 1,
    "name": "Libertino"
  },
  {
    "id": 2,
    "name": "Evaristo"
  }
]/ #
```

O docker-compose.yml

A estrutura do ficheiro para o caso prático Angular + json-server fica organizada em dois serviços com responsabilidades bem separadas:

- Serviço app;
- Serviço api.

O docker-compose.yml

- **Serviço app**: build a partir do Dockerfile do projeto Angular. Porta 4200 do host mapeada para 80 do contentor nginx. Dependência declarada em api.
- **Serviço api:** imagem pública clue/json-server. Volume monta o db.json local. Porta 3000 exposta apenas internamente; o host acede via 3001 para testes.

O docker-compose.yml

- Variável de ambiente JSON_SERVER_WATCH=1 ativa o modo de watch no json-server, que recarrega automaticamente ao detetar alterações no db.json.
- **Resultado**: docker compose up --build arranca os dois contentores, cria a rede interna, e a aplicação Angular em localhost:4200 já consome dados reais da API.

Recursos práticos – Consumir API JSON Server

```
import { environment } from '../environments/environment.development'

async function loadUsers() {
  const response = await fetch(environment.apiUrl + '/users')
  const users = await response.json()

  console.log(users)
}

loadUsers()
```

Innominatum CONSULTORIA

Docker Compose

ServiceNow - Deloitte

Prof. Tiago Rodrigues

IPCA POLITÉCNICO DO CÁVADO E DO AVE
INSTITUTO DO EMPREGO E FORMAÇÃO PROFISSIONAL
PRR Plano de Recuperação e Resiliência
REPÚBLICA PORTUGUESA
Financiado pela União Europeia NextGenerationEU
IAPMEI Parcerias para o Crescimento
PORTUGAL DIGITAL